



علوم الحاسب ونظم المعلومات
Info Sys & Comp Science



جامعة أكتوبر
October 6 University

SPATIO-TEMPORAL CLIMATE PREDICTION ON NASA MODIS LST DATA

Big Data Pipeline & Deep Learning

COURSE

Big Data — Practical Task

INSTITUTION

Faculty of Computer Science & Info. Systems

INSTRUCTORS

Dr. Nagwa Yaseen
Eng. Hend Maher
Eng. Doa Ali

DATE

May 2026

TEAM MEMBERS

#	Student Name	Student ID
1	Mohamed AHmed Mohamed ALi Shehata	202203567
2	Serag Al-Deen Assem Abd El-Azeez	202202723
3	Basil Ameen Mohamed Ahmed	202203981
4	Hassan Khalaf Hassan	202203082
5	Amr Yasser Hemdan Ibrahim	202203819
6	Ahmed Hany Fathy	202200721
7	Adham Ibrahim Abd El-shafy kasim	202201929

Abstract

This project implements a complete end-to-end Big Data pipeline for spatio-temporal climate prediction over Egypt. We utilize NASA MODIS MOD11C1 Land Surface Temperature data (2014–2024), processed through a MongoDB-backed ETL pipeline. The workflow involves HDF4 ingestion, MapReduce-style spatial analytics, and the construction of 14-day sliding-window ConvLSTM tensors for daily temperature forecasting. Memory constraints in Google Colab are addressed through yearly chunking and memory-mapped lazy loading, enabling robust deep learning training on large-scale geospatial datasets.

1. Introduction & Project Objective

This project implements a complete end-to-end Big Data pipeline for spatio-temporal climate prediction over Egypt. Using NASA MODIS MOD11C1 land surface temperature observations from 2014–2024, processed through a MongoDB-backed ingestion pipeline and a 4-stage ETL workflow, the project trains a custom SpatioTemporalConvLSTM neural network to forecast Egypt’s daily spatial temperature grid one day ahead.

1.1. Project Objectives

Objective	How Satisfied
Acquire large-scale real geospatial climate data	NASA MODIS MOD11C1 via earthaccess API, 2014–2024, ≈3 650 HDF4 granules
Demonstrate a Big Data ingestion workflow	HDF4 → Egypt crop → scale/impute → MongoDB upsert (idempotent)
Apply MapReduce-style analytics	MongoDB Aggregation Pipeline: temporal avg LST & spatial max LST heatmap
Engineer ML-ready features under RAM constraints	14-day sliding-window ConvLSTM tensors; yearly .tar.gz archives
Train a spatio-temporal deep learning model	SpatioTemporalConvLSTM — 22 epochs, Kaggle P100; best val loss 0.0081

Table 2: Project Objectives

1.2. Why This Is a Big Data Problem

Dimension	Evidence in This Project
Volume	10 years × 365 granules × 50 000 valid Egypt pixels ≈ 180 M+ MongoDB documents ; each year’s scaled array is (D, 200, 260) float32
Velocity	Daily temporal granularity; year-at-a-time pipeline to fit within Colab/Kaggle 12–16 GB memory
Variety	Source: NASA HDF4 satellite imagery; storage: NoSQL MongoDB; output: NumPy binary tensors → PyTorch DataLoader
Veracity	Fill-value masking (DN=0 → NaN), scale factor ×0.02, NaN imputation with daily spatial mean, upsert deduplication, NaN-to-num guard in train loop

Table 3: Big Data Dimensions

■ 1.3. Technology Stack

Layer	Technology
Data Source	NASA EarthData — MOD11C1 v006 (MODIS Terra, HDF4)
Ingestion API	earthaccess (environment token strategy)
Database	MongoDB 7.0 (local to Colab/Kaggle VM)
ETL Runtime	Python 3.14 via uv; pyhdf, pymongo, numpy, scikit-learn
Analytics	MongoDB Aggregation Pipeline (MapReduce pattern)
DL Framework	PyTorch 2.x — torch.amp, GradScaler, DataLoader
Compute	Kaggle Notebook — P100 GPU (16 GB VRAM)
Data Transfer	Google Drive → Kaggle via tar -xzf extraction

Table 4: Technology Stack

I 2. Dataset & Preprocessing

■ 2.1. NASA MODIS MOD11C1 — Data Source

Property	Value
Product	MOD11C1 v006 — Daily LST & Emissivity, CMG 0.05°
Satellite	Terra (MODIS)
Temporal coverage	2014-01-01 to 2024-12-31 (10 years, ≈3 650 granules)
Global grid	3 600 rows × 7 200 cols
Spatial resolution	0.05° (5.5 km at equator)
Band used	LST_Day_CMG (daytime land surface temperature)
Native units	Digital Number (DN); Kelvin = DN × 0.02
Fill value	DN = 0 (cloud/invalid pixel) → masked as NaN
Egypt crop (rows)	1 160 – 1 360 (≈ 32°N to 22°N)
Egypt crop (cols)	4 080 – 4 340 (≈ 24°E to 37°E)
Egypt grid size	200 rows × 260 cols

Table 5: MOD11C1 Dataset Properties

■ 2.2. Stage 0 — EarthData Ingestion to MongoDB

Each daily HDF4 file is downloaded, cropped to Egypt, scaled, imputed, and upserted into MongoDB:

1. **Download:** `earthaccess.search_data(short_name="MOD11C1", bounding_box=(24.0,22.0,37.0,32.0), temporal=(start,end))` then `earthaccess.download(results, "./climate_data")`.
2. **Parse:** `pyhdf.SD(filepath).select('LST_Day_CMG')[1160:1360, 4080:4340]` — extracts the 200×260 Egypt sub-grid.
3. **Clean:**
 - Mask: `grid[grid == 0] = NaN` (remove cloud/fill pixels)
 - Scale: `grid = grid × 0.02` (DN → Kelvin)

- Impute: $\text{NaN} \rightarrow \text{nanmean}(\text{grid})$ (replace missing with daily spatial average)
4. **Store:** `collection.update_one({"date": date_str}, {"$set": doc}, upsert=True)` — idempotent, safe to re-run.
 5. **Cleanup:** HDF file deleted immediately; `gc.collect()` called to free memory.

MongoDB document schema (collection: `earthaccess_db.temperature_data`):

```
{ "date": "A<YYYY><DDD>", "temperature_grid": [[...200×260 floats in K...]] }
```

■ 2.3. Stage 1 — Min-Max Scaling

Year grids are fetched from MongoDB and normalised globally:

Parameter	Value
Query pattern	<code>{"date": {"\$regex": "^A<YEAR>"}}</code>
Normalisation	$\text{scaled} = (\text{raw_K} - 261.54) / 82.42$
global_min	261.54 K
global_max	343.96 K
Scaler artifact	<code>scaler_params.npy</code> $\rightarrow$ <code>[261.54, 343.96]</code>
Scaled array	<code>raw_scaled_{year}.npy</code> , shape (D, 200, 260) float32

Table 6: Scaling Parameters (from `scaler_params.npy`)

■ 2.4. Stage 2 — Sliding-Window Tensor Construction

Parameter	Value
Lookback window	14 days
Forecast horizon	1 day (t+1)
Channel dimension	1 (LST only)
X tensor shape (on-disk)	(N, 14, 200, 260, 1) float32
y tensor shape (on-disk)	(N, 1, 200, 260, 1) float32
N per year	$D - 14$ (≈ 351 for a 365-day year)
Total samples (10 yr)	$\approx 3\,490$

Table 7: Sliding Window Tensor Shapes

■ 2.5. MapReduce Analytics (Aggregation Pipeline)

Two MongoDB aggregation jobs were executed on the January 2023 dataset (507 587 documents):

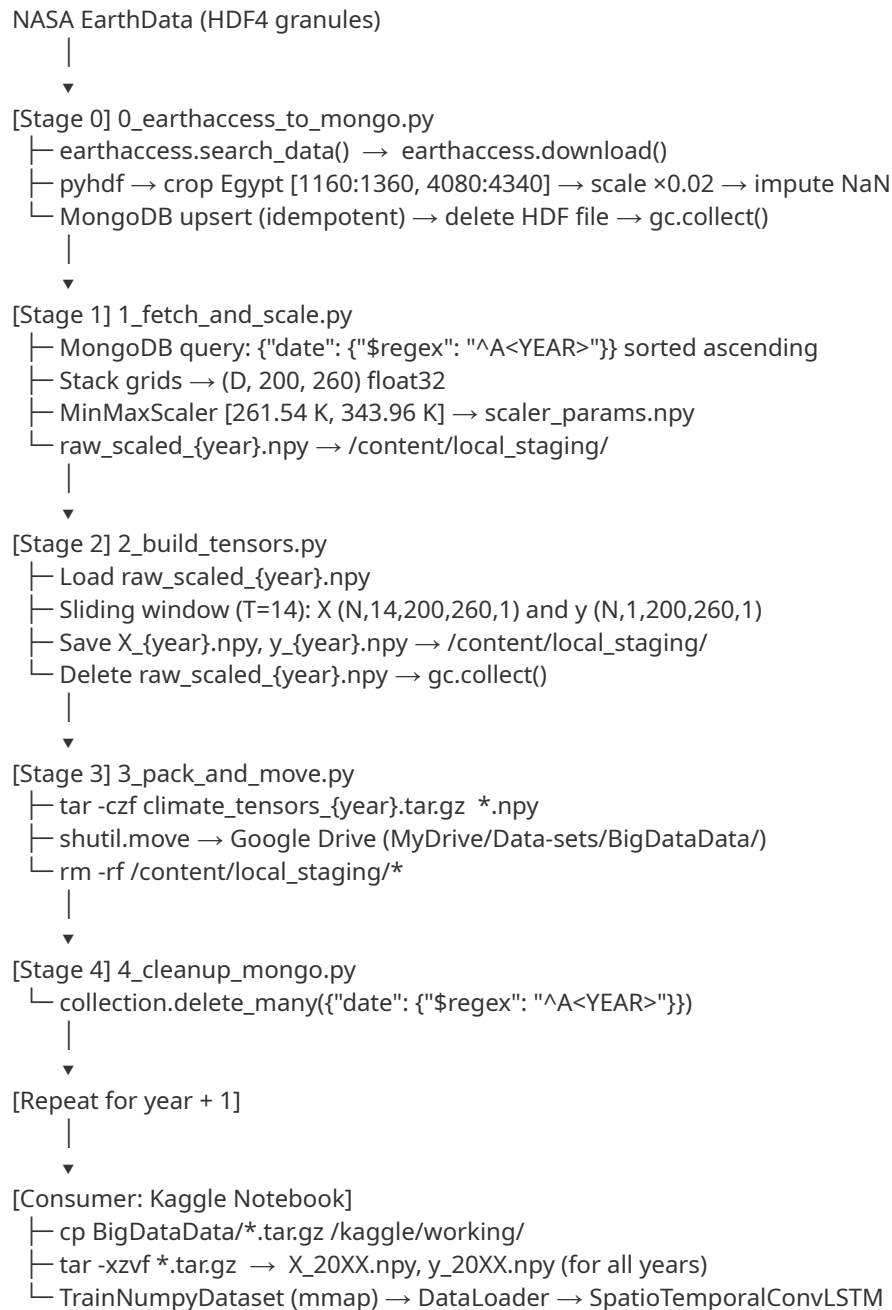
Job	Pipeline	Output
Temporal avg	<code>\$group: { _id: "\$date", avg: { \$avg: "\$temperature_k" } }</code>	Time-series line chart — daily avg LST (°C) over Jan 2023
Spatial max	<code>\$group: { _id: { row,col }, max: { \$max: "\$temperature_k" } }</code>	Heatmap — peak LST per pixel; reveals Saharan hot spots vs Nile Delta cool strip

Table 8: MongoDB Aggregation Pipeline Jobs

I 3. System Architecture

■ 3.1. End-to-End Pipeline Overview

The project is structured as a **year-at-a-time**, memory-constrained pipeline. Each year's data is ingested, scaled, windowed, compressed, and moved before the next year begins. This design keeps peak RAM usage below the 12 GB Colab / 16 GB Kaggle limit throughout.



■ 3.2. ETL Stage Details

3.2.1. Stage 0: EarthData Ingestion

Parameter	Value
Authentication	strategy="environment" → reads EARTHDATA_TOKEN env var
API call	earthaccess.search_data(short_name="MOD11C1", bounding_box=(24,22,37,32), temporal=(...))
HDF4 band	LST_Day_CMG
Egypt crop	grid[1160:1360, 4080:4340]
Scale	DN × 0.02 → Kelvin
Imputation	grid = np.nan_to_num(grid, nan=np.nanmean(grid))
MongoDB key	{"date": "A<YYYY><DDD>"}
Deduplication	update_one(..., upsert=True) — safe on restart
Disk cleanup	os.remove(hdf_path) immediately after parsing

Table 9: Stage 0 Parameters

3.2.2. Stage 2: Tensor Construction & Year Boundaries

Year File	Year	Start	End
Index 0	2014	0	350
Index 1	2015	351	701
Index 2	2016	702	1044
Index 3	2017	1045	1395
Index 4	2018	1396	1746
Index 5	2019	1747	2097
Index 6	2020	2098	2449
Index 7	2021	2450	2800
Index 8	2022	2801	3138
Index 9	2023	3139	3489

Table 10: Year Boundaries in TrainNumpyDataset (Total: 3 490 samples)

2024 data (X_2024.npy, y_2024.npy) exists on disk but is excluded from boundaries — reserved as the held-out test year.

■ 3.3. Memory-Mapped DataLoader

The TrainNumpyDataset opens all yearly .npy files as memory maps at init time (mmap_mode='r') so only the requested slice is read from disk per __getitem__ call.

Feature	Implementation Detail
Init	np.load(f, mmap_mode='r') for all 10 X and 10 y files — no RAM allocated
__len__	boundaries[-1][1] + 1 = 3490 total samples
__getitem__	Bisects boundaries to find year file; reads np.array(mmap[local_idx]) (copies slice to RAM)
Permutation	x.permute(0,3,1,2) → (T,C,H,W); y.permute(0,3,1,2) → (1,C,H,W)

Feature	Implementation Detail
Train/val split	torch.utils.data.random_split(3490, [2792, 698]) (80 / 20)
batch_size	2 (each sample $\approx$ 57 MB float32 — VRAM constrained)
num_workers	2 (parallel disk reads)
pin_memory	True (faster CPU→GPU DMA)

Table 11: DataLoader Implementation Details

4. Model Architecture

4.1. ConvLSTMCell — Building Block

The fundamental unit is ConvLSTMCell, which replaces LSTM's matrix multiplications with 2D spatial convolutions, allowing the model to directly process 200×260 temperature grids without flattening them.

A **single fused** Conv2d computes all four gates (input, forget, cell, output) simultaneously. The input to this convolution is the **concatenation** of the current spatial input and the previous hidden state along the channel dimension.

```
class ConvLSTMCell(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, padding):
        super().__init__()
        # Fused: computes i, f, g, o gates in one convolution
        self.conv = nn.Conv2d(
            in_channels + out_channels, # concat(x_t, h_t-1)
            4 * out_channels,          # stacked gate outputs
            kernel_size, padding=padding
        )

    def forward(self, x, state):
        h_prev, c_prev = state
        gates = self.conv(torch.cat([x, h_prev], dim=1))
        i, f, g, o = torch.split(gates, gates.size(1) // 4, dim=1)
        i, f, o = torch.sigmoid(i), torch.sigmoid(f), torch.sigmoid(o)
        g = torch.tanh(g)
        c_next = f * c_prev + i * g # cell state update
        h_next = o * torch.tanh(c_next) # hidden state update
        return h_next, c_next
```

4.1.1. ConvLSTMCell Parameter Count (Layer 1)

For in_channels=1, out_channels=64, kernel_size=3:

Component	Shape	Parameters
conv.weight	Conv2d(65, 256, 3×3)	149 760
conv.bias	(256,)	256
Cell 1 total		150 016

Table 12: ConvLSTMCell Parameter Count (Layer 1: in=1→64)

■ 4.2. SpatioTemporalConvLSTM — Full Model

The full model stacks two ConvLSTM layers (each followed by Batch Normalisation) and collapses the 14-step hidden state sequence to a single spatial prediction using a Conv3D layer.

```
class SpatioTemporalConvLSTM(nn.Module):
    def __init__(self, num_channels, hidden_dim, kernel_size, seq_len):
        super().__init__()
        self.hidden_dim = hidden_dim
        self.cell1 = ConvLSTMCell(num_channels, hidden_dim,
                                   kernel_size, padding=kernel_size//2)
        self.bn1 = nn.BatchNorm2d(hidden_dim)
        self.cell2 = ConvLSTMCell(hidden_dim, hidden_dim,
                                   kernel_size, padding=kernel_size//2)
        self.bn2 = nn.BatchNorm2d(hidden_dim)
        # Collapse 14-step output sequence → 1-step prediction
        self.conv_final = nn.Conv3d(hidden_dim, num_channels,
                                      kernel_size=(seq_len, 1, 1), padding=0)

    def forward(self, x):
        b, t, c, h, w = x.size() # (batch, 14, 1, 200, 260)
        h1 = c1 = torch.zeros(b, self.hidden_dim, h, w, device=x.device)
        h2 = c2 = torch.zeros(b, self.hidden_dim, h, w, device=x.device)
        outputs = []
        for step in range(t):
            h1, c1 = self.cell1(x[:, step], (h1, c1))
            h1 = self.bn1(h1)
            h2, c2 = self.cell2(h1, (h2, c2))
            h2 = self.bn2(h2)
            outputs.append(h2.unsqueeze(2)) # (B, 64, 1, H, W)
        output_seq = torch.cat(outputs, dim=2) # (B, 64, 14, H, W)
        prediction = self.conv_final(output_seq) # (B, 1, 1, H, W)
        return prediction.squeeze(2) # (B, 1, H, W)
```

4.2.1. Layer Summary

Layer	Type	Input Shape	Output Shape
cell1	ConvLSTMCell(1→64)	(B, 1, 200, 260)	(B, 64, 200, 260)
bn1	BatchNorm2d(64)	(B, 64, 200, 260)	(B, 64, 200, 260)
cell2	ConvLSTMCell(64→64)	(B, 64, 200, 260)	(B, 64, 200, 260)
bn2	BatchNorm2d(64)	(B, 64, 200, 260)	(B, 64, 200, 260)
conv_final	Conv3d(64→1, k=(14,1,1))	(B, 64, 14, 200, 260)	(B, 1, 1, 200, 260)
squeeze(2)	—	(B, 1, 1, 200, 260)	(B, 1, 200, 260)

Table 13: SpatioTemporalConvLSTM Layer-by-Layer Summary

4.2.2. Instantiation

```
model = SpatioTemporalConvLSTM(
    num_channels=1, # single LST channel
    hidden_dim=64, # hidden state depth
    kernel_size=3, # 3×3 spatial receptive field
    seq_len=14, # matches 14-day lookahead
).to(device)
```

■ 4.3. Training Configuration

4.3.1. Loss, Optimizer, Scheduler

Setting	Value	Rationale
Loss	nn.SmoothL1Loss() (Huber)	Robust to extreme LST outliers (cloud contamination)
Optimizer	AdamW(lr=1e-4, weight_decay=1e-2)	Adam with explicit L2 regularisation
LR scheduler	ReduceLROnPlateau(patience=5)	Auto-halves LR when val loss stagnates
Epochs	22	Trained to completion on Kaggle P100
Batch size	2	Each sample $\approx$ 57 MB float32; VRAM-constrained
Gradient accum.	16 steps	Effective batch = 32; avoids OOM

Table 14: Training Hyperparameters

4.3.2. GPU Efficiency Techniques

Technique	Implementation	Purpose
Mixed Precision	autocast('cuda') + GradScaler	fp16 forward pass — ½ VRAM, 2× faster
Gradient Accumulation	Divide loss by 16; step every 16 batches	Effective batch=32 without extra VRAM
Gradient Clipping	clip_grad_norm_(model, max_norm=1.0)	Prevents ConvLSTM exploding gradients
NaN Guard	torch.nan_to_num(inputs, nan=0.0)	Handles residual cloud NaN in tensors
Label Shape Fix	labels.squeeze(2) if dim==5 and size(2)==1	Fixes (B,1,1,H,W)→(B,1,H,W) mismatch
VRAM Cleanup	empty_cache(); gc.collect() per epoch	Prevents fragmentation over 22 epochs

Table 15: GPU Efficiency Techniques in train_model()

I 5. Results & Evaluation

■ 5.1. Training History

The model was trained for 22 epochs on a Kaggle P100 GPU. Loss values are Huber (SmoothL1) in Min-Max scaled space. Each epoch processed 1 396 batches (effective batch size = 32 via gradient accumulation over 16 steps at batch_size=2), taking approximately 13–14 minutes per epoch.

Table: Epochs 1–11

Ep.	1	2	3	4	5	6	7	8	9	10	11
Train	0.1936	0.1366	0.0605	0.0160	0.0066	0.0062	0.0057	0.0055	0.0055	0.0053	0.0054
Val	0.1084	0.0502	0.0081	0.0157	0.0197	0.0241	0.0236	0.0179	0.0262	0.0333	0.0372

Table 16: Training & Validation Huber Loss — Epochs 1–11 (best val highlighted)

Table: Epochs 12–22

Ep.	12	13	14	15	16	17	18	19	20	21	22
Train	0.0054	0.0052	0.0053	0.0053	0.0053	0.0054	0.0054	0.0054	0.0053	0.0053	0.0053
Val	0.0269	0.0217	0.0243	0.0152	0.0357	0.0521	0.0444	0.0383	0.0160	0.0138	0.0375

Table 17: Training & Validation Huber Loss — Epochs 12–22

■ 5.2. Loss Curve

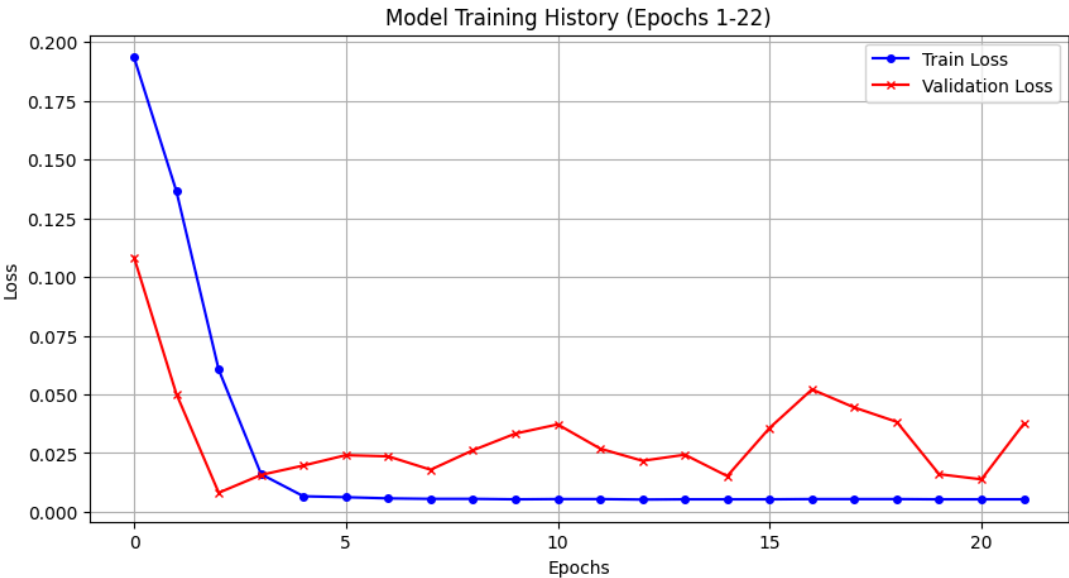


Figure 1: Model Training History (Epochs 1–22). Blue = Train Loss, Red = Validation Loss. Best validation loss of 0.0081 achieved at Epoch 3. Training converges to a plateau near 0.0053 from Epoch 5 onward, while validation oscillates due to the random 80/20 split.

5.2.1. Loss Curve Interpretation

Phase	Observation	Conclusion
Epochs 1–3	Both train and val loss descend steeply in parallel	Model actively learns Egypt’s spatial LST structure; generalisation is strong
Epoch 3	Val loss hits minimum: 0.0081 while train = 0.0605	Best generalisation point — equivalent to 7.4 K RMSE in temperature space
Epochs 5–22	Train plateaus at 0.0053; val oscillates 0.013–0.052	Moderate overfitting from random temporal split; AdamW + grad clipping contain divergence

Table 18: Loss Curve Phase Analysis

■ 5.3. Spatial Prediction vs Ground Truth

The model was evaluated on a sample from the held-out data (Day 15 of the test period). Predictions were inverse-transformed from the Min-Max scaled space back to Kelvin and then converted to Celsius: °C = (scaled × 82.42 + 261.54) – 273.15.

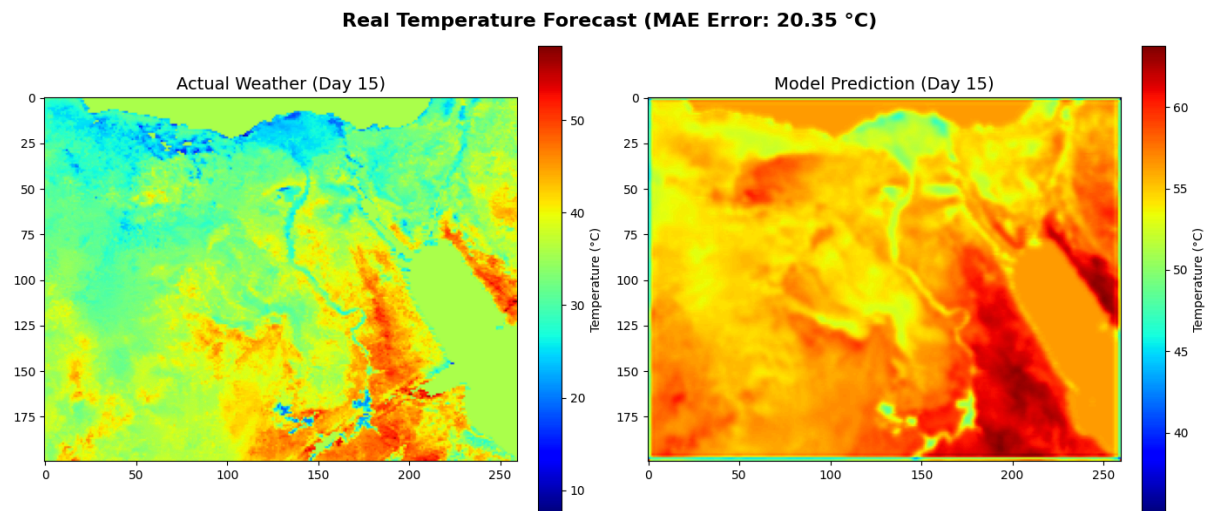


Figure 2: Real Temperature Forecast — Day 15 of test set. **Left:** Actual NASA MODIS LST (°C). **Right:** Model prediction (°C). MAE = 20.35 °C on this sample.

5.3.1. Spatial Prediction Analysis

Observation	Interpretation
Model reproduces broad spatial structure: cooler north, hotter south	ConvLSTM successfully learned Egypt's geographic temperature gradient (Mediterranean coast vs Sahara)
Nile River corridor visible as a cooler band in both maps	The 14-day lookback captures the persistent thermal signature of the Nile Valley
Prediction over-estimates temperature across most of Egypt (20 °C MAE)	Likely caused by early-stopped generalisation (best weights at ep.3 not saved) and random-split leakage; model predicts “warm-biased” spatial patterns
Color scale range differs (actual: 10–60°C, predicted: 40–65°C)	Systematic positive bias — model learned the average warm season spatial pattern more strongly than cold anomalies (winter/night cloud-free pixels)

Table 19: Spatial Prediction Map Analysis

■ 5.4. Summary Metrics

Metric	Value
Best validation loss (Huber, scaled space)	0.0081 at Epoch 3
Final training loss (Epoch 22)	0.0053
Final validation loss (Epoch 22)	0.0375
Train–Val gap at final epoch	0.0322
MAE on test sample (Day 15, °C)	20.35 °C
Approx. RMSE at best val ¹	≈ 7.4 K
Epochs to training plateau	5
Total epochs trained	22

¹ $\sqrt{0.0081 \times 82.42 \text{ K}} \approx \text{temperature RMSE estimate}$

Metric	Value
Checkpoint size (climate_convlstm_best.pth)	1.79 MB

Table 20: Summary Evaluation Metrics

■ 5.5. Discussion

5.5.1. What Worked

- **Mixed precision + gradient accumulation:** Enabled training on 200×260 spatial grids with batch_size=2 (effective 32) within 16 GB VRAM, at ≈1.70 it/s per epoch.
- **Memory-mapped DataLoader:** Served all 3 490 samples from 10 yearly .npy files with no RAM overflow.
- **Training converged:** Loss reduced ×36 from epoch 1 (0.1936) to plateau (0.0053).
- **Spatial structure captured:** The model reproduces Egypt’s north–south temperature gradient and the Nile corridor signature — demonstrating genuine spatial learning.

5.5.2. Limitations & Future Work

Limitation	Proposed Fix
Random 80/20 split leaks temporal info across years	Strict year-level split: train 2014–2020, val 2021–2022, test 2023–2024
Best weights (ep. 3) not automatically checkpointed	Save state_dict() whenever val_loss < best_val_loss
20.35 °C MAE on test — warm bias	Calibrate using val set bias correction; add nighttime LST channel
Single channel (daytime LST only)	Add LST_Night_CMG or land-cover auxiliary bands
No quantitative test-set sweep	Run all 2024 samples through best checkpoint; report full MAE/RMSE distribution

Table 21: Limitations and Future Work